Networkx: all Spanning Trees and their associated total weight

Asked 5 years, 11 months ago Modified 5 years, 11 months ago Viewed 3k times

1

5

Given a simple undirected grid network like this:

```
import networkx as nx
from pylab import *
import matplotlib.pyplot as plt
%pylab inline

ncols=3
N=3
G=nx.grid_2d_graph(N,N)
labels = dict( ((i,j), i + (N-1-j) * N ) for i, j in G.nodes() )
nx.relabel_nodes(G,labels,False)
inds=labels.keys()
vals=labels.values()
inds=[(N-j-1,N-i-1) for i,j in inds]
pos2=dict(zip(vals,inds))
nx.draw_networkx(G, pos=pos2, with_labels=True, node_size = 200,
node_color='orange',font_size=10)
plt.axis('off')
plt.title('grid')
plt.show()
```

And given that each edge has a weight corresponding to its length:

```
#Weights
from math import sqrt

weights = dict()
for source, target in G.edges():
    x1, y1 = pos2[source]
    x2, y2 = pos2[target]
    weights[(source, target)] = round((math.sqrt((x2-x1)**2 + (y2-y1)**2)),3)

for e in G.edges():
    G[e[0]][e[1]] = weights[e] #Assigning weights to G.edges()
```

How could it be possible to compute all spanning trees in the grid, and their associated total weight?

NB: this is a trivial case where all weights=1.

python networkx minimum-spanning-tree spanning-tree weighted-graph Edit tags

Share Edit Follow Close Flag

asked Oct 20, 2016 at 12:54



FaCoffee
7,221 26 91 166

1 I know that code.... ;-) Is the grid guaranteed to be regular? – Paul Brodersen Oct 20, 2016 at 13:52

Also, presumably you have seen that networkx has a function for the `minimum_spanning_tree` ? – Paul Brodersen Oct 20, 2016 at 13:52

Yes, you can plot it yourself to check. But your promising answer makes me think this is only possible for the grid network, correct? Because I'd have to do this for other network models as well :) – FaCoffee Oct 20, 2016 at 13:53

Yes I have seen and used the function, and it's not what I need. – FaCoffee Oct 20, 2016 at 13:54

I think for a regular k-grid enumerating all solutions wouldn't be too bad -- <k^|V| to be precise (if k is large, that is still bad, but at least you know that beforehand), and I am sure that you can leverage the fact that it is grid to get those. Implementing Prim's algorithm for the general case so that you get all spanning trees is also not impossible, just painful and the computation would be costly. – Paul Brodersen Oct 20, 2016 at 14:00

1 Answer

Sorted by:
Reset to default

Date modified (newest first)

5

This took way longer than expected, but the following code finds all spanning trees for the general case. Getting the associated total weight should be trivial, as you have access to the edgelist of each tree.

Don't use this on very large trees -- even the toy example yields 192 spanning trees.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import networkx as nx

def _expand(G, explored_nodes, explored_edges):
    """
    Expand existing solution by a process akin to BFS.

    Arguments:
    ----------
    G: networkx.Graph() instance
        full graph

    explored_nodes: set of ints
        nodes visited

    explored_edges: set of 2-tuples
        edges visited

    Returns:
    --------
    solutions: list, where each entry in turns contains two sets corresponding to
explored_nodes and explored_edges
        all possible expansions of explored_nodes and explored_edges

    """
    frontier_nodes = list()
    frontier_edges = list()
    for v in explored_nodes:
        for u in nx.neighbors(G,v):
            if not (u in explored_nodes):
                frontier_nodes.append(u)
                frontier_edges.append([(u,v), (v,u)])

    return zip([explored_nodes | frozenset([v]) for v in frontier_nodes],
[explored_edges | frozenset(e) for e in frontier_edges])

def find_all_spanning_trees(G, root=0):
    """
    Find all spanning trees of a Graph.

    Arguments:
    ----------
    G: networkx.Graph() instance
        full graph

    Returns:
    ST: list of networkx.Graph() instances
        list of all spanning trees

    """

    # initialise solution
    explored_nodes = frozenset([root])
    explored_edges = frozenset([])
    solutions = [(explored_nodes, explored_edges)]
    # we need to expand solutions number_of_nodes-1 times
    for ii in range(G.number_of_nodes()-1):
        # get all new solutions
        solutions = [_expand(G, nodes, edges) for (nodes, edges) in solutions]
        # flatten nested structure and get unique expansions
        solutions = set([item for sublist in solutions for item in sublist])

    return [nx.from_edgelist(edges) for (nodes, edges) in solutions]


if __name__ == "__main__":

    N = 3
    G = nx.grid_2d_graph(N,N)
    labels = dict( ((i,j), i + (N-1-j) * N ) for i, j in G.nodes() )
    nx.relabel_nodes(G,labels,False)
    inds=labels.keys()
    vals=labels.values()
    inds=[(N-j-1,N-i-1) for i,j in inds]
    pos2=dict(zip(vals,inds))

    fig, ax = plt.subplots(1,1)
    nx.draw_networkx(G, pos=pos2, with_labels=True, node_size = 200,
node_color='orange',font_size=10,ax=ax)
    plt.axis('off')
    plt.title('grid')

    ST = find_all_spanning_trees(G)
    print len(ST)

    for g in ST:
        fig, ax = plt.subplots(1,1)
        nx.draw_networkx(g, pos=pos2, with_labels=True, node_size = 200,
node_color='orange',font_size=10,ax=ax)
        plt.axis('off')
        plt.title('grid')
        plt.show()
```

Share Edit Follow Flag

edited Oct 20, 2016 at 16:41

answered Oct 20, 2016 at 16:32

Paul Brodersen
10.2k 19 36

I think the original version was working as intended but this definitely probably works. – Paul Brodersen Oct 20, 2016 at 16:46